Lab Worksheet 12
The Transport Layer

This Week's Objectives

1. Be able to configure multiple netkit labs.
2. Be able to describe the format of UDP packets.
3. Be able to describe the format of TCP packets.

Expected Outcomes

After the laboratory session, students should be able to:

- pass data from a UDP client to a UDP listener in Linux using the netcat (aka nc) instruction.
- analyse a captured pcap file using wireshark and explain the ports used in UDP datagrams.
- explain how UDP datagrams are carried in IP packets which are in turn carried over the LAN in Ethernet frames.
- adapt marginally incorrect instructions and / or Netkit configurations.
  - there will be some deliberate errors

Workplan

These lab worksheets assume that you will make accurate notes of everything that you do for later consideration and revision. You should use your lab diary to note down items such as commands used, the outcome of each command, and answers to questions posed in these instructions.

All lab machines have CherryTree Notes software installed, which facilitates organised note-taking. Alternatively, you could buy a hardback A4 laboratory notebook.

NB this is a complex lab – please read each instruction carefully – if you miss out any steps, then don't expect to achieve the correct results! You will need to have completed and thoroughly understood all the steps before the next lab session.

Remember that there is a lab work wiki available to you on BB. Please use this to share and communicate with your colleagues.

Make sure you have completed all the steps in this worksheet before the next lab session.

Lab Worksheet 12
The Transport Layer

Starting assumptions

1. You have completed all the work in previous lab sheets

   - You have used Netkit *lstart* to launch a coordinated group of several virtual machines. Specifically, you are aware of how this group is coordinated via the *lab.conf* file.
   - You know how to shut down virtual machines using *lhalt* and *lcrash*.
   - You have used *ip* and *ping* within the virtual machines.
   - You have saved captured network traffic from a virtual machine using *tcpdump*.
   - You have viewed a packet capture file in the real Linux host using *wireshark*.

Activities

Reminder

- You need to make accurate notes of what you do.
- You will complete unfinished activities before the next timetabled lab session.
- You will resolve what you do not understand by conducting your own careful (and ethically sound) experimentation and / or further reading.

Getting Started

2. Create a directory for lab's work. Use a command similar to

```
cd ~
mkdir -p ctec1704/lab_12
```

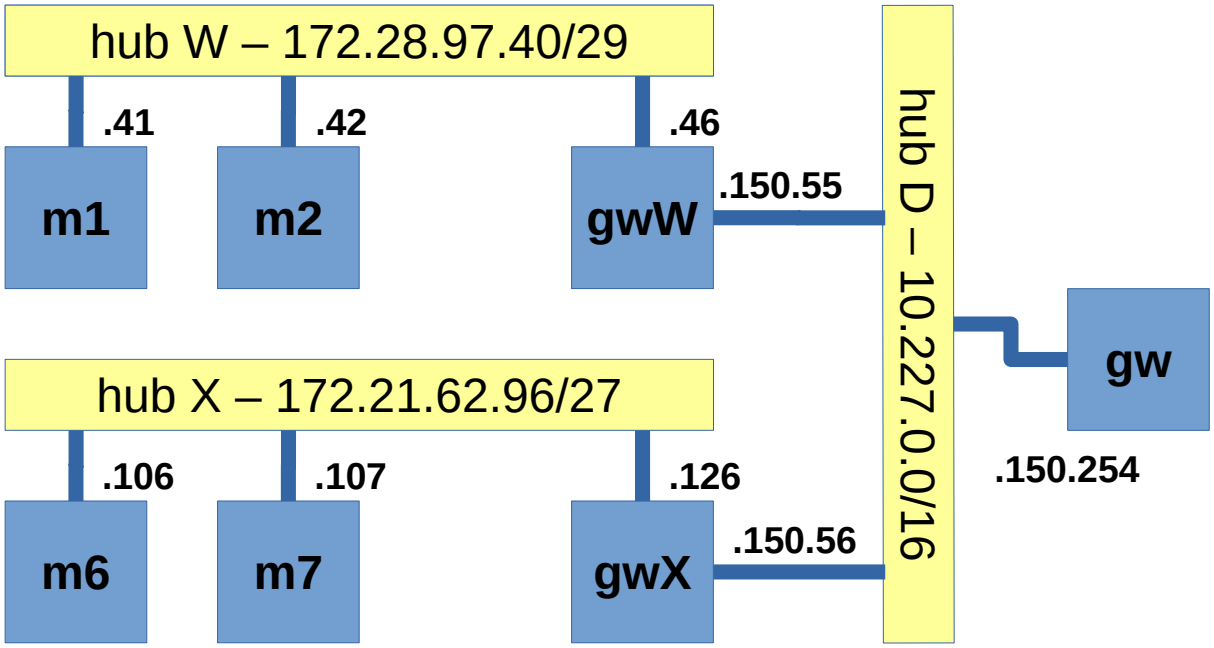
3. Download the zipped archive *netkit_lab_12.tar.xz* from Blackboard.
   - This contains the configuration information for virtual machines.

4. Extract the contents of the archive file, and make sure that all the files and directories for the lab are in the new lab_12 directory.

```
tar -xvf netkit_lab_12.tar.xz
```

5. This netkit lab represents the configuration of four hosts and three gateways / routers. Note that these are connected via hubs rather than switches.

   Look in the *lab.conf* file. Confirm that the ASCII art diagram corresponds to the diagram below (ignore the /29 and /27 for now):



6. Place yourself in the your newly created lab_12 directory and list its contents.

```
cd ~/ctec1704/lab_12
ls -l
```

7. As before, you will see several directories (some empty, some with content), one for each virtual machine.

8. Launch the virtual machines using the instruction:

```
lstart
```

9. Look carefully at the virtual machines as they start (be patient!). In particular record any errors you see.

10. Arrange the xterm windows on the screen so they are consistent with the network diagram above.

Look at the configuration

11. Once all machines are up, use tcpdump on machines *m2*, *m7* and *gw* to store captured traffic in the files *m2-d1.pcap*, *m7-d1.pcap* and *gw-d1.pcap* in a *captures* directory (create this directory, if you don't already have it). The instructions will be similar to the one below.

```
tcpdump -s0 -i eth0 -w /hosthome/ctec1704/captures/m2-d1.pcap
```

12. Make a careful note of the exact instruction you used on each machine.

TCP vs UDP

You are going to use the *netcat* command to create network traffic. *netcat* (nc) is known as the Swiss army knife of networking and uses either the TCP or UDP protocol to carry messages. Use the man page to read something about the tool, or read this brief tutorial.

Now generate some network traffic.

13. Netcat needs a *listener* to respond to incoming messages (TCP or UDP packets). On machine *gwW* launch the listener:
    - use the *man* command to understand the switches
    - convert the port number to hex and note the value

```
gwW:~# netcat -l -p 51966
```

14. From machine *m1*, send a TCP datagram to port *51966* of *gwW* (you have a choice of IP addresses - note down which you chose and why.):

```
m1:~# netcat ???.???.???.??? 51966
hello there
```

15. Text entered on *m1* will appear in the *gwW* xterm
    - *ctrl C* will stop netcat in each xterm.

16. Go back to *m1* and use netcat to send *hello* to *m*6 in a UDP datagram (don't forget to start a listener in m6):

```
echo "hello" | netcat ???.???.???.??? <<port>> -q0
```

17. Answer these questions:
    - what does the *-q* switch do in netcat?
    - what does the pipe | symbol do in the shell?
    - what does the *echo* command do?

18. Repeat the message transmissions using UDP packets – add the *-u* switch to both the listener and the sender.

19. Stop the packet capture in *m2* and look at its content in wireshark.

    - Make sure that you can find and explain the TCP and UDP packets in each of the activities above.
    - For each case, note which client port was used.
        - Is there a pattern?
    - Can you see the text messages you sent?
        - Hint – look in the hex pane (the lowest pane of the wireshark window)

20. Compare your results with those of a colleague and note similarities / differences.

21. Stop the other captures and open them in wireshark. What other packets have been sniffed (captured)?

Using the *screen* command

It is often useful to be able to look at two things concurrently in a netkit virtual machine. The *screen* command enables you to have several output terminals, only one of which is visible at a time. Think of them like a deck of cards. You can see the one that is on top and you can choose which one goes on top. Once you have started screen, *ctrl-a* followed by another character controls what screen does. For example,

- ctrl-a c will create another window (you get one for free when you start screen)
- ctrl-a n will move to the "next" window.
- ctrl-a p will move to the "previous" window.

22. We will use *screen* to do several things on machine m6.

23. First, start a packet capture in *m7* writing to a pcap file *m7-d1.pcap* (remember to send it to the *captures* directory).

```
tcpdump etc etc
```

24. Now on *m6*, start *screen* (read the initial messages, then press whatever is suggested to get to the shell prompt).

```
screen
```

25. On *m6* start a netcat listener on port *64206* (what is this in hex?).
NB *nc* is an *alias* for *netcat*

```
nc -u -l -p 64206
```

26. Create another window on *m6* using *screen.*

```
ctrl-a c
```

27. In the window you have just created, start another UDP listener on port *57005*.

```
nc -u -l -p 57005
```

28. Check you can switch between the two windows on *m6*:

```
ctrl-a n
ctrl-a n
```

29. Create a third window on *m6*:

```
ctrl-a c
```

30. In the window you have just created, start another UDP listener, this time on port 51966:

```
nc -u -l -p 51966
```

31. Create a fourth widow on *m6*:

```
ctrl-a c
```

We will use another common network tool to inspect the state of the listeners – *netstat*. Read more about this useful command here.

32. In this fourth window launch *netstat* to look for all IPv4 packets (this excludes Unix sockets from the display since these do not interest us):

```
netstat -an4
```

It would be useful if we could have a continuous view of the network status via netstat rather than the one-shot view. There is a *-c* option to continuously refresh the output of the watch command every second. This however causes the display to scroll quite vigorously.

33. Start the watch command on *m6*:

```
watch -n1 netstat -an4
```

34. Keep this *netstat* output as the visible screen on *m6*.

35. Now, on *m1*, *m2* and *gwW*, connect to the three UDP listeners you have running on *m6*.

    - on *m1*, you should force the use of client port 11111 to connect to 64206
    - on *m2* use client port 22222 to connect to port 57005
    - on *gwW* let the operating system choose the client port to connect to port 51966.

    Do these need to be different or could they be the same? For example, on *m1* this would be:

```
echo "m1m1m1" | nc -u 172.21.62.106 64206 -p 11111 -q0
```

36. Record what *netstat* shows as you send each datagram.

37. Stop the packet capture on *m7* and inspect the pcap file with *wireshark*. Explain the port number in each UDP datagram.

Make sure you have completed all the steps in this worksheet before the next lab session.